Lesson 10:
Graphics Softwares and Introduction to OpenGL/WebGL

Graphics Software

- There are two types of graphics software
  - Special purpose packages
    - For non programmers
    - The interface is a set of menus that facilitate communication
    - E.g. CAD, CAM, painting, animation
  - General programming packages
    - Provides a library of graphics functions that can be used in a programming language such as C, C++, Java, …
    - Library of graphics functions (OpenGL)
      - Picture components (lines, shapes, …)
      - Transformations (color, texture, shading, rotating, …)

Coordinate Representations

- To generate a picture using a programming package, we first need to give the geometric descriptions of the objects – location and shape
- For example, a box is specified by the positions of its corners, a sphere is specified by its center position and radius
- General graphics packages require geometric descriptions to be specified in a standard, right-handed, Cartesian-coordinate reference frame.

Graphics Functions

- Graphics Output Primitives (building blocks)
  - Character strings, lines, filled color areas (usually polygons)
  - Basic tools for constructing pictures
- Attributes
  - Color, line style, text style, area filling patterns
- Geometric Transformations
  - Change size, position or orientation of an object

Algorithms

- A number of basic algorithms are needed:
- Transformation: Convert representations of models/primitives from one coordinate system to another
- Clipping/Hidden surface removal: remove primitives and part of primitives that are not visible on the display
- Rasterization: Convert a projected screen space primitive to a set of pixels.
- Picking: select a 3D object by clicking an input device over a pixel location.
- Shading and illumination: Simulate the interaction of light with a scene.
- Animation: Simulate movement by rendering a sequence of frames.

Early History of APIs

- The primary goal of standardized graphics software is portability
- The first graphics API was
  - Graphical Kernel System (GKS)
    - 2D but contained good workstation model
  - GKS adopted as IS0 and later ANSI standard (1984)
- GKS not easily extended to 3D (GKS-3D)
  - Far behind hardware development

PHIGS and X

- Programmers Hierarchical Graphics System (PHIGS)
  - Arose from CAD community in 1988
  - Database model with retained graphics (structures)
- X Window System
  - DEC/MIT effort
  - Client-server architecture with graphics
- PEX combined the two
  - Not easy to use (all the defects of each)
- Phigs+, GL, OpenGL, Direct3D: 3D pipeline

OpenGL

- Silicon Graphics, Inc. (SGI) developed graphics workstations with a set of routines called GL (Graphics Library),
- The success of GL lead to OpenGL (1992), a platform-independent API that was
  - Easy to use
  - Close enough to the hardware to get excellent performance
  - Focus on rendering
  - Omitted windowing and input to avoid window system dependencies

OpenGL Evolution

- Originally controlled by an Architectural Review Board (ARB)
  - Members included SGI, Microsoft, Nvidia, HP, 3DLabs, IBM,…….
  - Relatively stable (present version 4.6)
    - Evolution reflects new hardware capabilities
      - 3D texture mapping and texture objects
      - Vertex programs
  - Allows for platform specific features through extensions
  - ARB replaced by Kronos

OpenGL Libraries

- OpenGL core library
  - OpenGL32 on Windows
  - GL on most unix/linux systems (libGL.a)
- OpenGL Utility Library (GLU)
  - Provides functionality in OpenGL core but avoids having to rewrite code
- Links with window system
  - GLX for X window systems
  - WGL for Windows
  - AGL for Macintosh

Basic OpenGL Syntax

- Function names are prefixed with gl for core library, glu for GLU, glut for GLUT library
  - glBegin, glClear, gluOrtho2D,glutInit
- Constants
  - GL_2D, GL_RGB, GLUT_SINGLE
- Data types
  - GLbyte, GLshort, GLint, GLfloat, GLdouble

GLUT

- OpenGL Utility Toolkit (GLUT)
  - Provides functionality common to all window systems
    - Open a window
    - Get input from mouse and keyboard
    - Menus
    - Event-driven
  - Code is portable but GLUT lacks the functionality of a good toolkit for a specific platform
    - No slide bars

OpenGL Functions

- Primitives
  - Points
  - Line Segments
  - Polygons
- Attributes
- Transformations
  - Viewing
  - Modeling
- Control (GLUT)
- Input (GLUT)
- Query

OpenGL State

- OpenGL is a state machine
- OpenGL functions are of two types
  - Primitive generating
    - Can cause output if primitive is visible
    - How vertices are processed and appearance of primitive are controlled by the state
  - State changing
    - Transformation functions
    - Attribute functions

Lack of Object Orientation

- OpenGL is not object oriented so that there are multiple functions for a given logical function
  - **`glVertex3f`**
  - **`glVertex2i`**
  - **`glVertex3dv`**
- Underlying storage mode is the same
- Easy to create overloaded functions in C++ but issue is efficiency

OpenGL function format

function name

dimensions

```
glVertex3f(x,y,z)
```

belongs to GL library

x,y,z are `floats`

```
glVertex3fv(p)
```

p is a pointer to an array

OpenGL Command Formats

glVertex3fv(v)

Number of components

2 - (x,y)
3 - (x,y,z)
4 - (x,y,z,w)

Data Type

b - byte
ub - unsigned byte
s - short
us - unsigned short
i - int
ui - unsigned int
f - float
d - double

Vector

omit "v" for scalar form

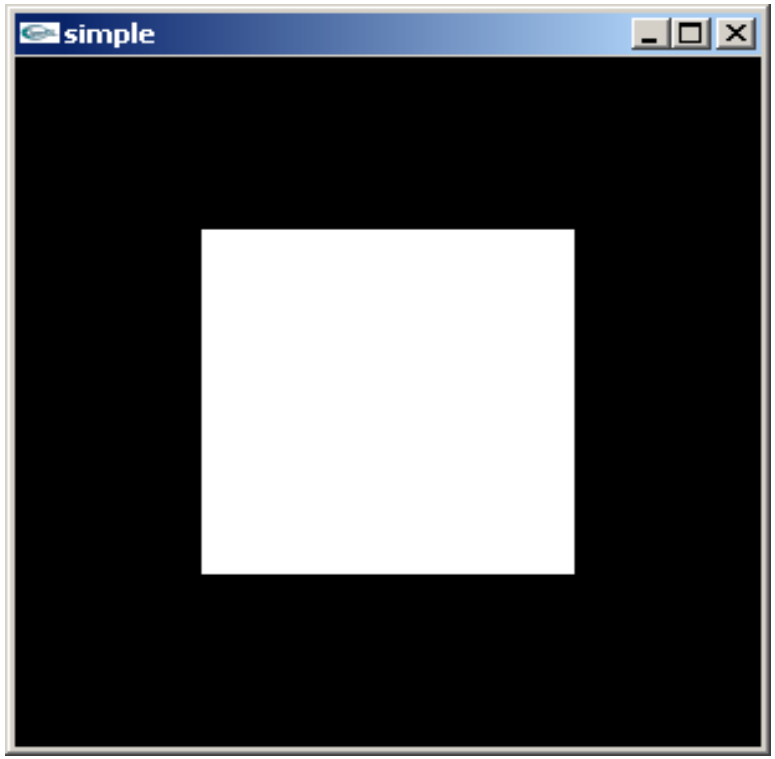
glVertex2f(x, y)

OpenGL #defines

- Most constants are defined in the include files **`gl.h`**, **`glu.h`** and **`glut.h`**
  - Note **`#include <GL/glut.h>`** should automatically include the others
  - Examples
  - **`glBegin(GL_POLYGON)`**
  - **`glClear(GL_COLOR_BUFFER_BIT)`**
- include files also define OpenGL data types: **`GLfloat`**, **`GLdouble`**,….

A Simple Program

Generate a square on a solid background



simple.c

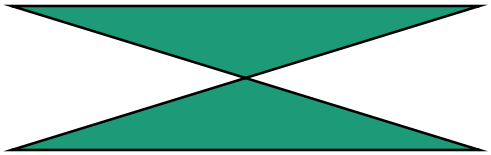
```
#include <GL/glut.h>
void mydisplay(){
     glClear(GL_COLOR_BUFFER_BIT);
   glBegin(GL_POLYGON);
      glVertex2f(-0.5, -0.5);
      glVertex2f(-0.5, 0.5);
      glVertex2f(0.5, 0.5);
      glVertex2f(0.5, -0.5);
   glEnd();
   glFlush();
}
int main(int argc, char** argv){
   glutCreateWindow("simple");
   glutDisplayFunc(mydisplay);
   glutMainLoop();
}
```

Compilation on Windows

- Visual C++
  - Get glut.h, glut32.lib and glut32.dll from web
  - Create a console application
  - Add opengl32.lib, glut32.lib to project settings (under link tab)
- Borland C similar
- Cygwin (linux under Windows)
  - Can use gcc and similar makefile to linux
  - Use –lopengl32 –lglu32 –lglut32 flags

Polygon Issues

- OpenGL will only display polygons correctly that are
  - Simple: edges cannot cross
  - Convex: All points on line segment between two points in a polygon are also in the polygon
  - Flat: all vertices are in the same plane
- User program can check if above true
  - OpenGL will produce output if these conditions are violated but it may not be what is desired
- Triangles satisfy all conditions



nonsimple polygon

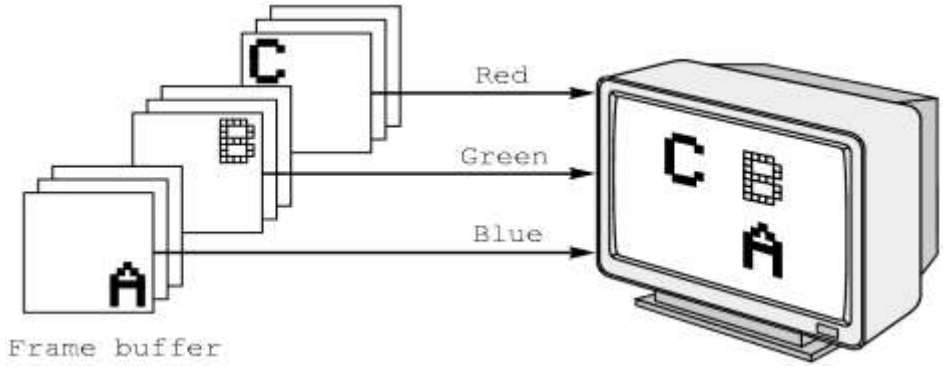
nonconvex polygon

Attributes

- Attributes are part of the OpenGL state and determine the appearance of objects
  - Color (points, lines, polygons)
  - Size and width (points, lines)
  - Stipple pattern (lines, polygons)
  - Polygon mode
    - Display as filled: solid color or stipple pattern
    - Display edges
    - Display vertices

RGB color

- Each color component is stored separately in the frame buffer
- Usually 8 bits per component in buffer
- Note in **`glColor3f`** the color values range from 0.0 (none) to 1.0 (all), whereas in **`glColor3ub`** the values range from 0 to 255



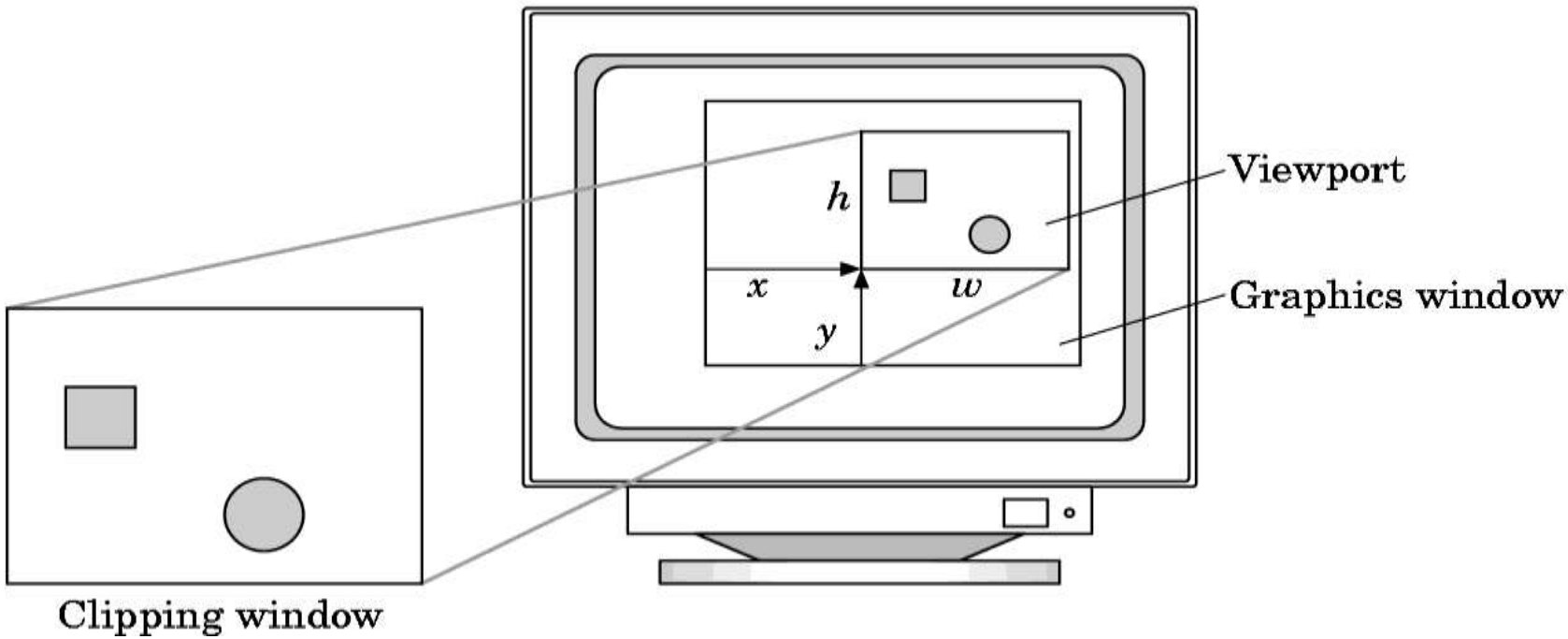
Color and State

- The color as set by **`glColor`** becomes part of the state and will be used until changed
  - Colors and other attributes are not part of the object but are assigned when the object is rendered
- We can create conceptual *vertex colors* by code such as

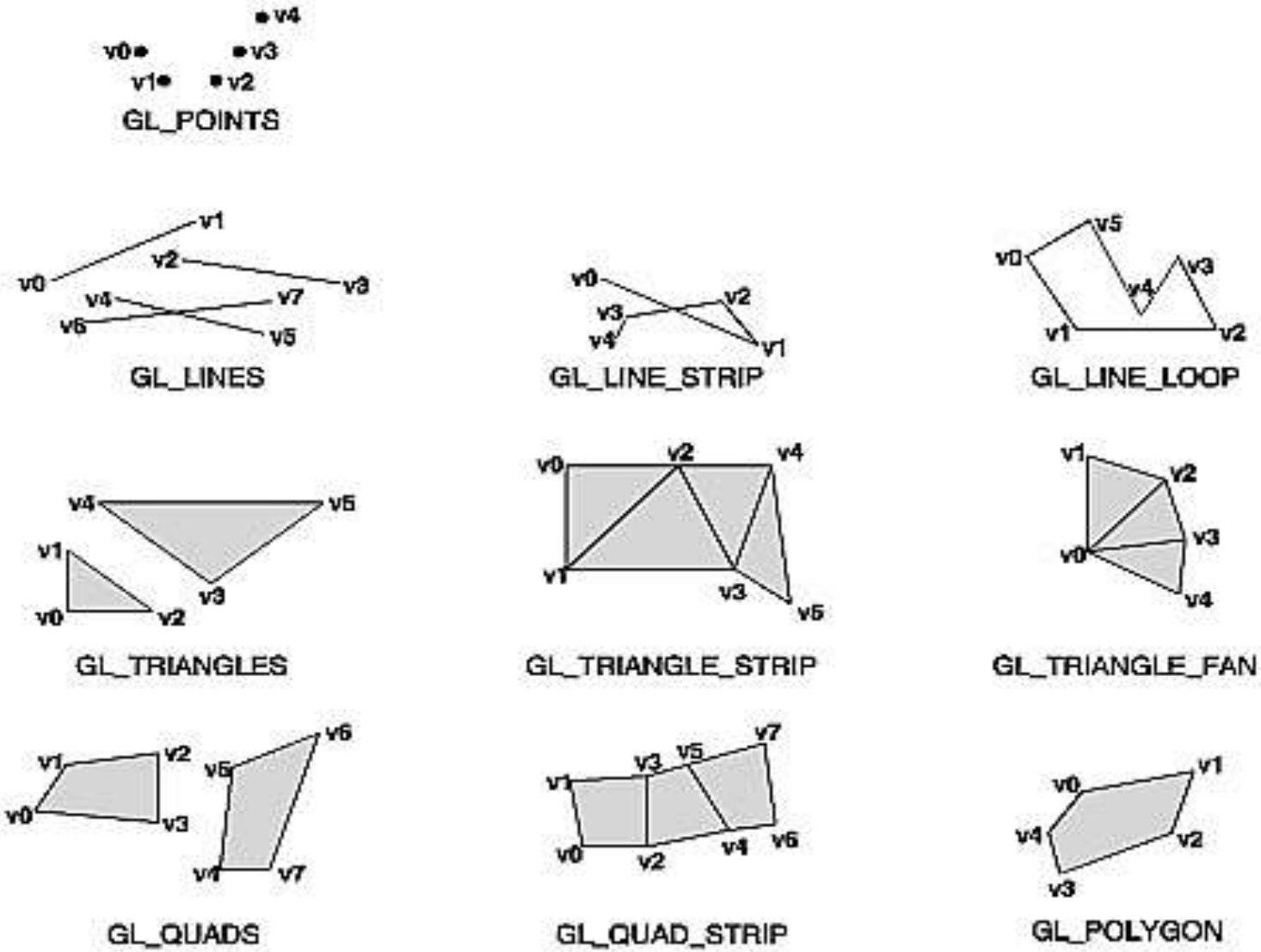
```
glColor
  glVertex
  glColor
  glVertex
```

Viewports

- Do not have to use the entire window for the image: **`glViewport(x,y,w,h)`**
- Values in pixels (screen coordinates)



Geometric Primitive Types



Geometric Primitive Types

- GL_POINTS : Draws a point at each of the *n* vertices.
- GL_LINES : Draws a series of unconnected line segments. Segments are drawn between v0 and v1, between v2 and v3, and so on. If *n* is odd, the last segment is drawn between vn-3 and vn-2, and vn-1 is ignored.
- GL_LINE_STRIP : Draws a line segment from v0 to v1, then from v1 to v2, and so on, finally drawing the segment from vn-2 to vn-1. Thus, a total of *n-1* line segments are drawn. Nothing is drawn unless *n* is larger than 1. There are no restrictions on the vertices describing a line strip (or a line loop); the lines can intersect arbitrarily.
- GL_LINE_LOOP : Same as GL_LINE_STRIP, except that a final line segment is drawn from vn-1 to v0, completing a loop.

Geometric Primitive Types

- GL_TRIANGLES : Draws a series of triangles (three-sided polygons) using vertices v0, v1, v2, then v3, v4, v5, and so on. If *n* isn't an exact multiple of 3, the final one or two vertices are ignored.
- GL_TRIANGLE_STRIP : Draws a series of triangles (three-sided polygons) using vertices v0, v1, v2, then v2, v1, v3 (note the order), then v2, v3, v4, and so on. The ordering is to ensure that the triangles are all drawn with the same orientation so that the strip can correctly form part of a surface. Preserving the orientation is important for some operations, such as culling.
- GL_TRIANGLE_FAN :Same as GL_TRIANGLE_STRIP, except that the vertices are v0, v1, v2, then v0, v2, v3, then v0, v3, v4, and so on.

Geometric Primitive Types

- GL_QUADS : Draws a series of quadrilaterals (four-sided polygons) using vertices v0, v1, v2, v3, then v4, v5, v6, v7, and so on. If *n* isn't a multiple of 4, the final one, two, or three vertices are ignored.
- GL_QUAD_STRIP: Draws a series of quadrilaterals (four-sided polygons) beginning with v0, v1, v3, v2, then v2, v3, v5, v4, then v4, v5, v7, v6, and so on. *n* must be at least 4 before anything is drawn. If *n* is odd, the final vertex is ignored.
- GL_POLYGON :Draws a polygon using the points v0, ... , vn-1 as vertices. *n* must be at least 3, or nothing is drawn. In addition, the polygon specified must not intersect itself and must be convex. If the vertices don't satisfy these conditions, the results are unpredictable.

Header files

- #include <windows.h>
- #include <GL/gl.h>
- #include <GL/glu.h>

With GLUT we don’t need glu.h and gl.h

- #include <GL/glut.h>
- Include header files for the C program
- #include <stdio.h>
- #include <stdlib.h>
- #include <math.h>

DISPLAY WINDOW MANAGEMENT USING GLUT

- To initialize glut------glutInit(&argc, argv)
- Window caption----glutCreateWindow("My first opengl program")
- To specify what the display window will contain you can use a method for example “ line_segment”
- Use glutDisplayFunc(line_segment) to call the method in the main program

COMPLETE THE OPERATIONS

- To complete the processing operations, use the following function------glutMainLoop()---the very last function of the program.
- Initial image position---- glutInitWindowposition(100,200)
- Initial window size----- glutInitWindowSize(200,300)
- Other window options like buffering and choice of color modes
- glutDisplayMode(GLUT_SINGLE|GLUT_RGB)

A complete OpenGL program

- Set background colour ---glClearColor(1.0,1.0,1.0,1.0)
- Putting the display window on the screen use---glClear(GL_COLOR_BUFFER_BIT)
- Choosing colour schemes for objects---glColor3f(1.0,0.0,0.0)

CONT…

- If we want to display 2D object, we tell the OpenGL how we want to project our picture onto the display window as generating 2D is a special case of 3D viewing
- glMatrixMode(GL_PROJECTION)
- gluOrtho2D(0.0,200.0,0.0,150.0)

CONT..

- Create “line_segement” by calling appropriate OpenGL routines
- glBegin(GL_LINES);
- glVertex2i(180,15);
- glVertex2i(10,145);
- glEnd();
- glFlush();

IMPLEMENTATION

- // Include Header files
- #include<windows.h>
- #include<GL/glut.h>
- #include<stdio.h>
- #include <stdlib.h>
- #include <math.h>

/*Initial set up */

- **void init (void)**
- **{**
- **glClearColor (1.0, 1.0, 1.0, 0.0); // Set display-//window color to white.**
- **glMatrixMode (GL_PROJECTION); // Set //projection parameters.**
- **gluOrtho2D (0.0, 200.0, 0.0, 150.0);**
- **}**

/*Creating a line*/

- **void lineSegment (void)**
- **{**
- **glClear (GL_COLOR_BUFFER_BIT); // Clear display window.**
- **glColor3f (1.0, 0.0, 0.0); // Set line segment color to red.**
- **glBegin (GL_LINES);**
- **glVertex2i (180, 15); // Specify line-segment geometry.**
- **glVertex2i (10, 145);**
- **glEnd ( );**
- **glFlush ( ); // Process all OpenGL routines as quickly as possible.**
- **}**

/*The main function*/

- **void main (int argc, char** argv)**
- **{**
- **glutInit (&argc, argv); // Initialize GLUT.**
- **glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB); // Set display mode.**
- **glutInitWindowPosition (50, 100); // Set top-left display-window position.**
- **glutInitWindowSize (400, 300); // Set display-window width and height.**
- **glutCreateWindow ("An Example OpenGL Program"); // Create display window.**
- **init ( ); // Execute initialization procedure.**
- **glutDisplayFunc (lineSegment); // Send graphics to display window.**
- **glutMainLoop ( ); // Display everything and wait.**
- **}**

/*More procedures*/

```
void line_segment(void)
{
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(0.0,0.0,1.0);
glBegin(GL_LINE_STRIP);
glVertex2i(180,15);
glVertex2i(10,145);
glVertex2i(100,15);
glVertex2i(10,185);
glEnd();
glFlush();
}
```

Cont...

```
void line_segment(void)
{
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(0.0,0.0,1.0);
glBegin(GL_LINE_LOOP);
glVertex2i(180,15);
glVertex2i(10,145);
glVertex2i(100,15);
glVertex2i(10,185);
glEnd();
glFlush();
}
```

OpenGL Example

- Here's an example that renders a colored triangle (Note that this doesn't include any 'setup' code)

```
glBegin(GL_TRIANGLES);
glColor3f(1.0, 0.0, 0.0);   // red
glVertex3f(-4.0, -2.0, 0.0);
glColor3f(0.0, 1.0, 0.0);   // green
glVertex3f(4.0, -2.0, 0.0);
glColor3f(0.0, 0.0, 1.0);   // blue
glVertex3f(0.0, 5.0, 0.0);
glEnd();
```

WebGL

- WebGL is abbreviated as Web Graphics Library.
- It is mainly designed for rendering 2D graphics and Interactive 3D graphics.
- It is Javascript API that can be used with HTML5.
- It supports cross-platform, and it is available in the English language only.
- The WebGL programs consist of a control code that is written in JavaScript.
- *OpenGL is called as Open Graphics Library. It is referred to as a cross-language and platform application programming interface for rendering two dimensional and three-dimensional vector graphics. OpenGL provides many functionalities like extensions.*

Revision Questions

1. Differentiate between GLU and GLUT components of an OpenGL.
2. Explain the following OpenGL functions:
   a) glVertex2i(12,30)
   b) glClearColor (1.0,1.0,0.7,0.1)
   c) glDisplayMode(object)
3. Compare and Contrast WebGL and OpenGL